

Prototype Simple Traditional Chatbot Using Python

Introduction

Chatbots have become an important part of human-computer interaction. Nowadays, companies use chatbots. Some chatbots use Artificial Intelligence and Large Language Models, while others use simple predefined rules.

Instead of using Artificial Intelligence, the chatbot responds based on predefined keywords and if-else conditions. The main objective of this project was to understand the basic working process of a chatbot and learn how humans and computers communicate using simple conversation rules.

This project also helped me understand the chatbot development lifecycle, testing process, and basic software development concepts. The chatbot was designed to answer common questions such as greetings, date, time, help, and exit commands.

Chatbot Lifecycle

Plan

I decided to build a simple chatbot that could communicate with users using predefined rules. Since I was new to chatbot development, I selected a rule-based approach because it is easier to understand and implement.

The chatbot was planned to answer simple questions instead of complex conversations. The goal was to build a working chatbot that demonstrates the basic concepts of human-computer interaction.

Requirement Analysis

Before writing the code, I identified the basic requirements for the chatbot.

The chatbot should:

- Welcome the user.
- Respond to greeting messages.
- Tell its name.
- Display available commands.
- Show the current date.
- Show the current time.
- End the conversation politely.
- Respond politely when it does not understand the user's question.

These requirements helped me organize the chatbot before starting development.

Design

After collecting the requirements, I designed the conversation flow.

The chatbot waits for the user's input. It converts the text into lowercase letters and compares it with predefined keywords. If the keyword matches, the chatbot displays the corresponding response. If there is no matching keyword, it returns a default message saying that it could not understand the question.

The design is simple and easy to understand because it uses rule-based decision making instead of Artificial Intelligence.

Development

I used if-else statements to compare the user's input with predefined keywords.

Python's built-in datetime module was used to display the current date and time. Since only Python standard libraries were used, no external packages were required.

The chatbot can respond to greetings, provide its name, display help information, show the current date and time, and exit the conversation when the user types "bye".

This approach follows the traditional chatbot model because every response is already defined in the program.

Test

The following scenarios were tested:

- Greeting messages such as "Hi" and "Hello".
- Asking the chatbot's name.
- Displaying the help menu.
- Showing the current date.
- Showing the current time.
- Entering an unknown question.
- Exiting the chatbot.

The chatbot responded correctly for all predefined commands. When an unknown question was entered, it displayed a polite message indicating that it could not understand the input.

Maintenance

Even though this chatbot is simple, it can be improved in the future.

- Adding more predefined questions.
- Connecting the chatbot with a database.
- Supporting voice input.
- Using Natural Language Processing.
- Integrating cloud-based AI services.

These improvements can make the chatbot more useful and interactive.

Implementation

The chatbot was implemented as a console-based Python application.

- chatbot.py
- README.md
- requirements.txt

The chatbot starts by displaying a welcome message. It then waits for user input and compares the entered text with predefined conditions.

It displays a default message informing the user that the question is not recognized.

This implementation helped me understand how traditional chatbots work without using machine learning or Artificial Intelligence.

Results

The chatbot successfully completed the expected functionality.

It was able to:

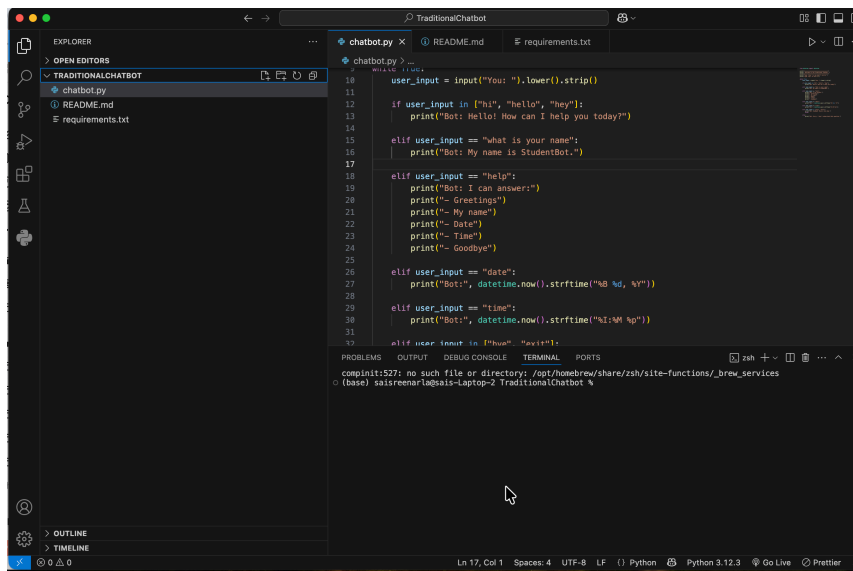
- Welcome users.

- Answer greeting messages.
- Show available commands.
- Display the current date.
- Display the current time.
- Handle invalid questions.
- Exit the conversation correctly.

The chatbot worked as expected during testing.

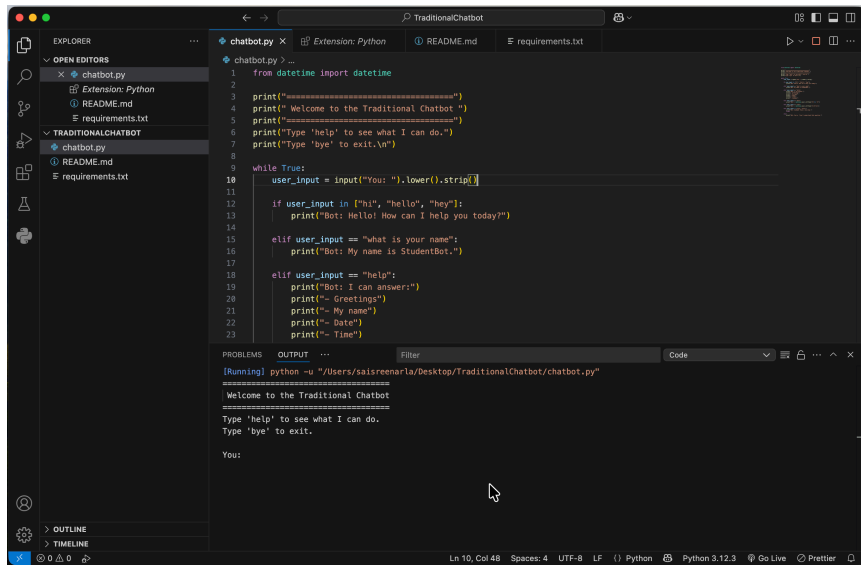
Screenshots

Figure 1. Project Structure



The figure shows the project folder containing the chatbot source code, README file, and requirements file.

Figure 2. Chatbot Startup



The chatbot starts successfully and displays the welcome message along with basic instructions.

Figure 3. Response

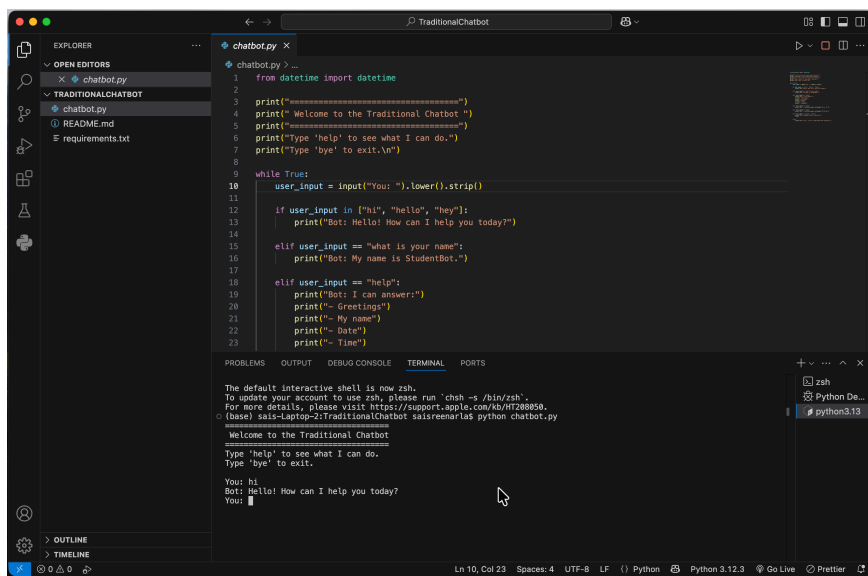


Figure 4. Help Command

The screenshot shows a Visual Studio Code editor with a file explorer on the left. The file explorer shows a project named 'TRADITIONALCHATBOT' containing 'chatbot.py', 'README.md', and 'requirements.txt'. The main editor window displays the code for 'chatbot.py'. The code imports the 'datetime' module and uses a 'while True' loop to handle user input. It checks for 'hi', 'hello', 'hey' (responds with 'Hello! How can I help you today?'), 'what is your name' (responds with 'My name is StudentBot.'), and 'help' (responds with a list of supported commands: 'Greetings', 'My name', 'Date', 'Time', 'Goodbye'). The terminal at the bottom shows the execution of 'python chatbot.py' and the interaction with the chatbot.

```
1 from datetime import datetime
2
3 print("=====")
4 print(" Welcome to the Traditional Chatbot ")
5 print("=====")
6 print("Type 'help' to see what I can do.")
7 print("Type 'bye' to exit.\n")
8
9 while True:
10     user_input = input("You: ").lower().strip()
11
12     if user_input in ["hi", "hello", "hey"]:
13         print("Bot: Hello! How can I help you today?")
14
15     elif user_input == "what is your name":
16         print("Bot: My name is StudentBot.")
17
18     elif user_input == "help":
19         print("Bot: I can answer:")
20         print("- Greetings")
21         print("- My name")
22         print("- Date")
23         print("- Time")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

The default interactive shell is now zsh.
To update your account to use zsh, please run 'chsh -s /bin/zsh'.
For more details, please visit <https://support.apple.com/kb/HT208850>.
(base) saia-Laptop-2:TraditionalChatbot saia\$ python chatbot.py

Welcome to the Traditional Chatbot

Type 'help' to see what I can do.
Type 'bye' to exit.

You: hi
Bot: Hello! How can I help you today?
You: help
Bot: I can answer:
- Greetings
- My name
- Date
- Time
- Goodbye
You:

Ln 10, Col 23 Spaces: 4 UTF-8 LF () Python Python 3.12.3 Go Live Prettier

The chatbot displays the list of supported commands.

Figure 5. Date and Time

This screenshot is similar to the previous one, but the terminal shows the chatbot's response to the 'time' command. The code in 'chatbot.py' is the same. The terminal output shows the user entering 'time', and the bot responding with the current time: '07:05 PM'.

```
1 from datetime import datetime
2
3 print("=====")
4 print(" Welcome to the Traditional Chatbot ")
5 print("=====")
6 print("Type 'help' to see what I can do.")
7 print("Type 'bye' to exit.\n")
8
9 while True:
10     user_input = input("You: ").lower().strip()
11
12     if user_input in ["hi", "hello", "hey"]:
13         print("Bot: Hello! How can I help you today?")
14
15     elif user_input == "what is your name":
16         print("Bot: My name is StudentBot.")
17
18     elif user_input == "help":
19         print("Bot: I can answer:")
20         print("- Greetings")
21         print("- My name")
22         print("- Date")
23         print("- Time")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

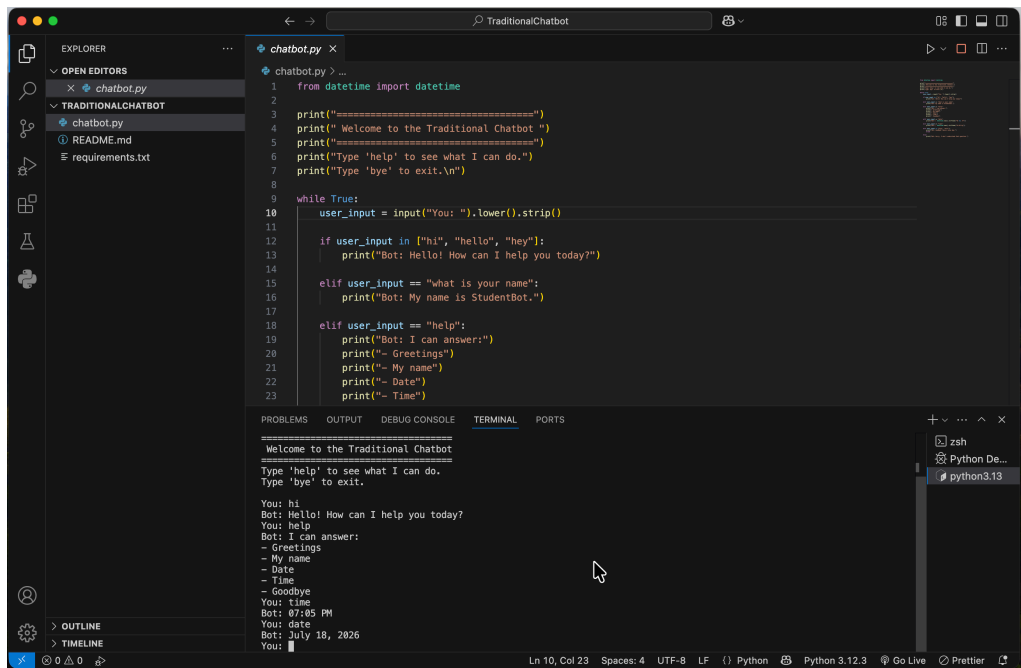
For more details, please visit <https://support.apple.com/kb/HT208850>.
(base) saia-Laptop-2:TraditionalChatbot saia\$ python chatbot.py

Welcome to the Traditional Chatbot

Type 'help' to see what I can do.
Type 'bye' to exit.

You: hi
Bot: Hello! How can I help you today?
You: help
Bot: I can answer:
- Greetings
- My name
- Date
- Time
- Goodbye
You: time
Bot: 07:05 PM
You:

Ln 10, Col 23 Spaces: 4 UTF-8 LF () Python Python 3.12.3 Go Live Prettier



```
1 from datetime import datetime
2
3 print("=====")
4 print(" Welcome to the Traditional Chatbot ")
5 print("=====")
6 print("Type 'help' to see what I can do.")
7 print("Type 'bye' to exit.\n")
8
9 while True:
10     user_input = input("You: ").lower().strip()
11
12     if user_input in ["hi", "hello", "hey"]:
13         print("Bot: Hello! How can I help you today?")
14
15     elif user_input == "what is your name":
16         print("Bot: My name is StudentBot.")
17
18     elif user_input == "help":
19         print("Bot: I can answer:")
20         print("- Greetings")
21         print("- My name")
22         print("- Date")
23         print("- Time")
```

=====

Welcome to the Traditional Chatbot

Type 'help' to see what I can do.

Type 'bye' to exit.

You: hi

Bot: Hello! How can I help you today?

You: help

Bot: I can answer:

- Greetings
- My name
- Date
- Time
- Goodbye

You: time

Bot: 07:05 PM

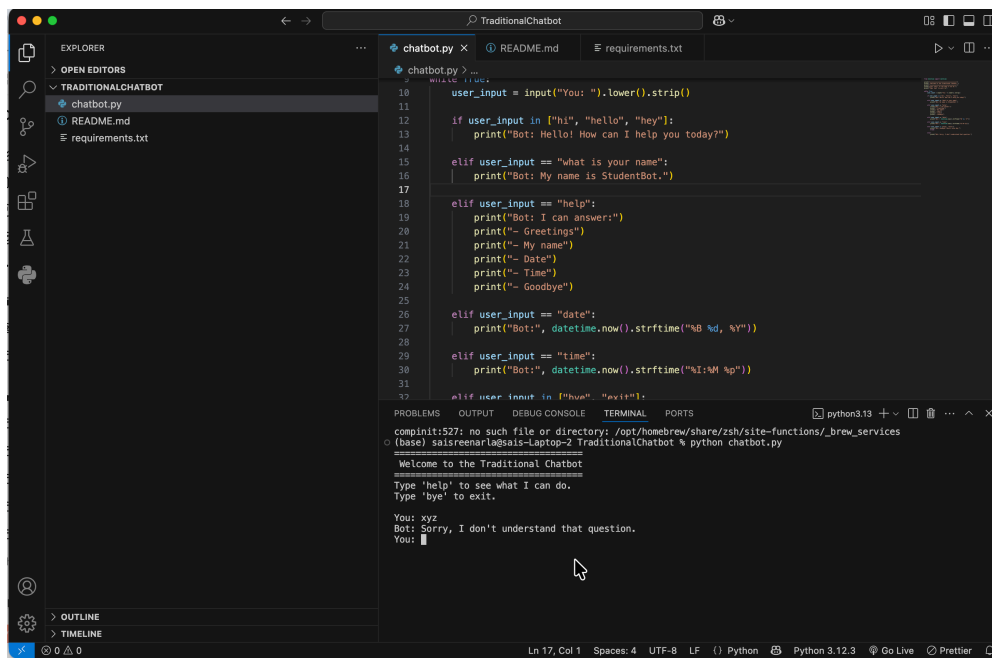
You: date

Bot: July 18, 2026

You:

The chatbot retrieves and displays the current system date and time.

Figure 6. Invalid Input



```
10 while True:
11     user_input = input("You: ").lower().strip()
12
13     if user_input in ["hi", "hello", "hey"]:
14         print("Bot: Hello! How can I help you today?")
15
16     elif user_input == "what is your name":
17         print("Bot: My name is StudentBot.")
18
19     elif user_input == "help":
20         print("Bot: I can answer:")
21         print("- Greetings")
22         print("- My name")
23         print("- Date")
24         print("- Time")
25         print("- Goodbye")
26
27     elif user_input == "date":
28         print("Bot:", datetime.now().strftime("%B %d, %Y"))
29
30     elif user_input == "time":
31         print("Bot:", datetime.now().strftime("%I:%M %p"))
32
33     elif user_input in ["bye", "exit"]:
34         break
```

=====

Welcome to the Traditional Chatbot

Type 'help' to see what I can do.

Type 'bye' to exit.

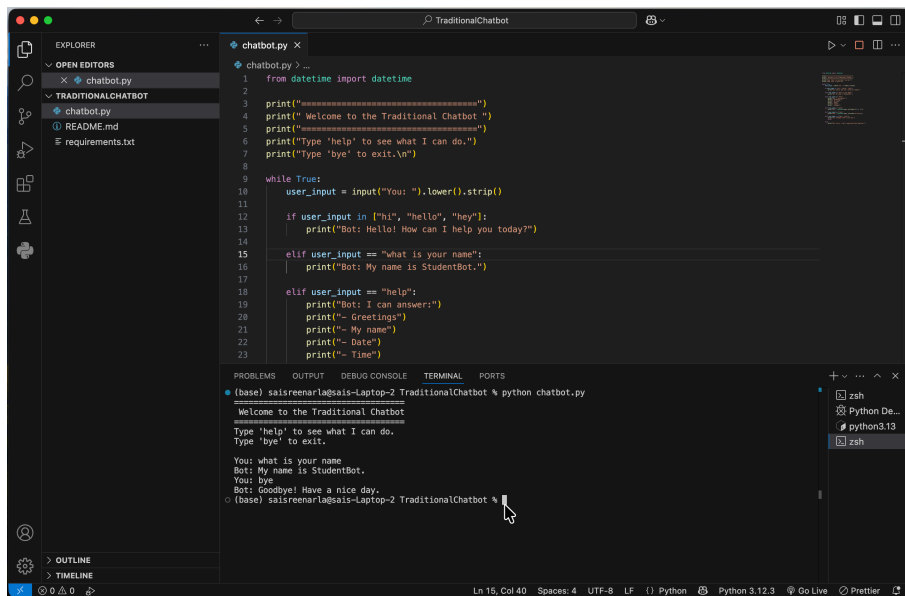
You: xyz

Bot: Sorry, I don't understand that question.

You:

The chatbot politely informs the user that it does not understand the entered question.

Figure 7. Exit Command



The screenshot shows a VS Code editor with a file explorer on the left containing 'chatbot.py', 'TRADITIONALCHATBOT', 'README.md', and 'requirements.txt'. The main editor window displays the 'chatbot.py' script. The script imports 'datetime', prints a welcome message, and enters a 'while True' loop. Inside the loop, it takes user input and responds based on keywords: 'hi', 'hello', 'hey' (greeting), 'what is your name?' (name), 'help' (help), and 'bye' (exit). The terminal at the bottom shows the execution of 'python chatbot.py', with the chatbot's responses to the user's inputs. The user enters 'bye', and the chatbot prints 'Type 'bye' to exit.' and then 'Goodbye! Have a nice day.' before exiting the loop.

```
1 from datetime import datetime
2
3 print("=====")
4 print(" Welcome to the Traditional Chatbot ")
5 print("=====")
6 print("Type 'help' to see what I can do.")
7 print("Type 'bye' to exit.\n")
8
9 while True:
10     user_input = input("You: ").lower().strip()
11
12     if user_input in ["hi", "hello", "hey"]:
13         print("Bot: Hello! How can I help you today?")
14
15     elif user_input == "what is your name?":
16         print("Bot: My name is StudentBot.")
17
18     elif user_input == "help":
19         print("Bot: I can answer:")
20         print("- Greetings")
21         print("- My name")
22         print("- Date")
23         print("- Time")
```

Terminal Output:

```
(base) saireenar@saais-Laptop-2: TraditionalChatbot % python chatbot.py
Welcome to the Traditional Chatbot
Type 'help' to see what I can do.
Type 'bye' to exit.

You: what is your name
Bot: My name is StudentBot.
You: bye
Bot: Goodbye! Have a nice day.
(base) saireenar@saais-Laptop-2: TraditionalChatbot %
```

The chatbot ends the conversation and displays a goodbye message.

Challenges

Initially, I was confused about the difference between a traditional chatbot and an AI chatbot. After reading the course material, I understood that a traditional chatbot works based on predefined rules instead of machine learning.

Another challenge was writing the conversation rules correctly. Sometimes the chatbot did not recognize the user's input because the entered text did not exactly match the predefined keyword. I corrected this by testing different inputs and improving the conditions.

I also learned how to organize the project files, upload the project to GitHub, and create the ZIP file required for submission.

These challenges helped me improve my understanding of chatbot development and debugging.

What I Learned

This project helped me learn many new concepts.

I learned how keyword matching works using if-else statements.

I also learned how to test a chatbot using different user inputs and how to manage the project using GitHub.

Overall, this project improved my confidence in developing simple chatbot applications using Python.

GitHub repository URL: <https://github.com/saisreen/Traditional-Rule-Based-Chatbot>

Conclusion

Although the chatbot is simple, it successfully demonstrates how a computer can communicate with users using predefined conversation rules.

The project also helped me understand software development, testing, debugging, and project documentation. In the future, I would like to improve this chatbot by adding Natural Language Processing, database support, and cloud-based AI services to make it more intelligent and interactive.

References

Python (n.d.). *Python documentation*. <https://docs.python.org/3/>